Lab 3 — Context, Completions, and Custom Instructions

After Module 3 — Copilot Features and Context

Open the TodoApp folder in VS Code. Confirm git status is clean before you start.

Part A — Inline completions

Step 1 — Comment-driven completion

Open your router file, **backend/routers/todos.py** or whatever it is called in your project.

Go to the end of the file. Type this comment and press Enter:

```
# GET /todos/stats - return total count, completed count, and pending count
```

Wait for the grey suggestion to appear. Press Tab to accept. If nothing appears, press Alt + \ to trigger a suggestion manually.

Shortcuts you will need: Tab accepts, Ctrl + Right Arrow accepts one word, Esc dismisses, Alt +] and Alt + [cycle alternatives, Alt + \ triggers manually.

Step 2 — The open-tabs effect

Close every editor tab except the router file. Delete the stats function body and trigger a suggestion. Note what it produces.

Now open **backend/repository/todo_repository.py** and **backend/models/todo.py** in tabs alongside it. Return to the router, delete the body again, and trigger a suggestion.

Compare the two suggestions. The second should reference your actual repository methods and field names. Nothing about the file changed — only what was open.

Accept the better suggestion and keep it.

Part B — Next Edit Suggestions

Step 3 — Rename a field and follow the arrows

Open backend/models/todo.py. Find the field IsCompleted and rename it to Completed. Type the change by hand.

An arrow appears in the gutter. Press Tab to jump to the next place that needs changing. Press Tab again to accept the edit there.

Keep pressing Tab — jump, accept, jump, accept — until no more arrows appear. Copilot should walk you through the repository, the service, the router, and the React component.

Step 4 — Discard the rename

Run the application. If it breaks, note which reference NES did not catch. Then discard everything:

```
git restore .
```

The rename was an exercise, not a change we are keeping. Confirm with **git status** that the working tree is clean, then re-add the stats endpoint from Step 1 if it was discarded.

Part C — Repository indexing and semantic search

Step 5 — Ask about the project without naming any file

Set the mode picker to Ask. Run these two prompts one at a time. Do not use #file or #codebase.

```
Describe the architecture of this project. What are the layers and how
do they depend on each other?
```

```
If I wanted to add a new field to a todo, which files would I have to
change and in what order?
```

Copilot was never told which files to read — the repository index found them by meaning. Expand the References list at the top of either answer and note that it included files you never opened.

Step 6 — Compare three ways of giving context

Run these three prompts in order and compare the answers.

```
How is data stored in this application?
```

```
#codebase How is data stored in this application?
```

```
#file:backend/repository/todo_repository.py How is data stored in this
application?
```

The first relies on the index. The second forces a semantic search across the workspace. The third narrows to one file. Note which answer was most precise and which was fastest.

Part D — Custom Instructions

Copilot has been guessing your conventions. Now you will tell it. You will create three files: one global, one for the backend, one for the frontend.

Step 7 — Create the three instruction files

Create .github/copilot-instructions.md:

```
# Copilot Instructions - TodoApp
```

Project

A single-user Todo application. No authentication. Built in versions:

- Version 1 (current): in-memory storage, no database
- Version 2: SQLite via SQLAlchemy with Alembic migrations
- Version 3: Category field added to todos

Repository layout

Monorepo. `backend/` is Python FastAPI. `frontend/` is React with Vite.

`docs/` holds PRD.md and TRD.md, which are the source of truth.

Architecture

Three backend layers, strictly separated:

- routers - HTTP concerns only. No business logic. No data access.
- services - business rules. Calls the repository. Never touches HTTP.
- repository - the only code that stores or retrieves data.

When storage changes, only the repository layer changes.

Conventions

- Python: snake_case, 4-space indent, type hints on all function signatures.
- JavaScript: camelCase, 2-space indent, functional components only.
- Todo fields are Id, Title, Description, DateCreated, IsCompleted. Use these names exactly, in both the API and the frontend.

Do

- Keep code simple and readable. Prefer clarity over cleverness.
- Return correct HTTP status codes: 201 on create, 204 on delete, 404 when a todo id does not exist.
- Consult docs/TRD.md before proposing structural changes.

Do NOT

- Add authentication, authorization, or user accounts.
- Add a database, ORM, or Category field until the relevant version.

```
- Add new dependencies without stating why.
- Put data access code in a router or a service.
- Use TypeScript. This project is JavaScript.
```

Create **.github/instructions/backend.instructions.md:**

```
---
applyTo: "backend/**"
---

# Backend Instructions

## Stack
Python 3.12, FastAPI, Pydantic. No ORM in Version 1.

## Rules
- Every endpoint is defined in a router file. Routers call services only.
- Services contain the business rules and never import FastAPI.
- The repository holds the in-memory list. It is the only module that
  knows how todos are stored.
- Request and response bodies are Pydantic models. Do not return raw
  dictionaries from an endpoint.
- Ids are generated in the repository, not in the router or service.
- Use `datetime.now(timezone.utc)` for DateCreated. Never naive
  datetimes.

## Do NOT
- Import the repository directly into a router.
- Use global mutable state outside the repository module.
- Add try/except blocks around normal control flow. Let FastAPI handle
  what it handles.
```

Create **.github/instructions/frontend.instructions.md:**

```
---
applyTo: "frontend/**"
---
```

```
# Frontend Instructions

## Stack
React 18 with Vite. JavaScript, not TypeScript. Function components
with hooks.

## Rules
- One component per file. The file name matches the component name.
- State lives in the highest component that needs it, and is passed
  down as props. No global state library in Version 1.
- All API calls go through a single module, `src/api/todoApi.js`.
  Components never call fetch directly.
- The backend base URL comes from an environment variable, never a
  hardcoded string in a component.
- Field names from the API are used as-is: Id, Title, Description,
  DateCreated, IsCompleted.

## Do NOT
- Add a UI component library or a CSS framework.
- Add routing. Version 1 is a single page.
- Use class components.
```

Step 8 — Verify all three files load

Open Copilot Chat and ask any question, for example:

```
How should I add a new endpoint to this project?
```

Expand the References list on the answer. You should see copilot-instructions.md listed. Ask a question that clearly concerns backend code and check that backend.instructions.md also appears. If they do not appear, press Ctrl + comma, search for instruction file, and confirm the setting is enabled. Also confirm the files are in .github/ at the repository root.

Step 9 — See the difference

Switch the chat mode to Agent. Run this:

```
Add an endpoint that returns only the completed todos.
```

Read the generated code and check three things: is the data access in the repository rather than the router, does the response use a Pydantic model, and are the field names exactly as specified? Compare this against how code was generated in Lab 2, before any instructions existed.

Step 10 — Commit

```
git add .
git commit -m "Add custom instructions, stats and completed endpoints"
git push
```

What you should have at the end of this lab

```
TodoApp/
  .github/
    copilot-instructions.md
    instructions/
      backend.instructions.md
      frontend.instructions.md
```

Plus two new endpoints, and a working understanding of what Copilot could see before you told it anything.